

S:

1. Review the capacity planning issues that are:
 1. In the Open column of the board.
 1. Previously assessed but now and therefore in need of a new look.
 - Use the labels auto-populated on the issue to help prioritize if necessary.
violation:hard is more important than violation:soft,
and severity:: provides additional signal.
1. For each saturation issue that is up for review,
evaluate the prediction given in the issue:
 - Check the tips below for suggestions on quickly assessing predictions as false positives;
this is a good way to quickly reduce the number of issues requiring more work
 - If evaluation requires non-trivial investigation over more than a day calendar time,
label it capacity-planning:investigate and investigate when you have dealt with higher
priority capacity planning issues
 - If you assess that it warrants active action in the near future and is not already
capacity-planning::in-progress:
 1. Label it ~capacity-planning:in-progress,
 1. Add or update the due date to next week, and
 1. Create a remediation issue
 - Take into consideration whether we have existing remediation options
(for example: performance-based overlays, or entire reference architecture upsizing)
or if we will need to add capabilities to handle the particular saturation problem.
Err on the side of raising an issue for further discussion;
we can always close it and return to monitoring status.
 - If it's in capacity-planning::in-progress check on the remediation issue and ensure it is
making progress,
and update the due-date to be 1 week in the future.
If remediation has completed,
move the issue to capacity-planning::verification
or, if results are already clearly sufficient, close it.
 - If it is in capacity-planning::verification,
check if the remediation results can be considered sufficient,
and if so close the issue.
If not, update the due date to next week for further review.
 - If the prediction has a wide-range and there is no indication that it will breach any time
soon,
or the lead time is sufficient that there is no urgency (for example 3+ months for Gitaly
disk saturation),
label the issue with ~capacity-planning::monitor
and update the due-date to just before the start of the next shift
for review by the incoming duty engineer.
 - If a metric is of a nature (perhaps for reasons specific to Dedicated)
that predictions are consistently unusable across all customers for that metric,
or if the prediction is plausibly useful but needs tuning:
 1. Label it capacity-planning::tune-model,
 1. Update the due-date to 2 weeks in the future,
 1. Work on the tamland
manifest
to exclude or tweak the specific saturation signal.

- The Observability team

can offer advice on the finer details of the tamland configuration.

1. Check that Tamland is running.

The pipeline should run successfully every day.

Investigate and fix any errors or failures.

1. Check if there are new production tenants not listed.

Update the list as necessary and create corresponding tenant:: labels.

When your shift comes to an end,

create a handover issue

assign it to the incoming duty engineer and populate with any information that the incoming shift should know about.

Let comments/discussions on the specific issues speak for mundane and routine matters, to be reviewed on their due date, but consider noting:

1. Any metrics that were coming close to being a concern but didn't warrant remediation just yet,

or that are in some way unusual

2. Brief comment on any ongoing remediations.

Reassign the remediation implementation issues to the incoming duty engineer,

unless you want to finish them up yourself or they are assigned to someone else for specific reasons.

Remember to record updates and status comments on each capacity-planning issue when necessary or useful,

just like you would on a team issue.

Some tips:

1. Forecasts may take several months to become predictable on new tenants.

As long as the literal numbers aren't too high,

do not be alarmed if the predicted range is wide and tamland is being overly cautious.

Putting into capacity-planning::monitor state is a good course of action in this situation

1. Sometimes Tamland will generate alerts for services whose saturation forecast is generally trending downwards,

but which Tamland's confidence interval (the light blue area in prediction graphs) still includes the possibility of saturation.

Unless you have specific reasons to suspect a real saturation risk,

strongly consider tagging the issue as capacity-planning::monitor and moving its due date out by 2+ weeks.

1. Try to get ahead of obvious gradual growth with enough time to take calm action.

For example, 1 month of warning for something like Gitaly or Opensearch disk usage is

sufficient to schedule an expansion of the storage volumes during upcoming maintenance windows,

not in a hurry in response to a pager alert

1. For items that need to be monitored it is encouraged to attach the current forecast in the comment;

the forecast will likely change in place over the following weeks, and the history can be useful.

1. Look for the component / alert name in the

Capacity Planning Issue Tracker can be a good source of information,

as some recurring saturation forecast share the same or similar causes,
or just to gain some insight into how these issues have been investigated and resolved in the past.

1. Remember that some items may alert across multiple tenants.

If possible, treat them as a unit rather than per-tenant,
unless the causes and fixes are truly distinct.

This is particularly important for false-positives or items needing tuning

1. Trust your instincts.

If it looks concerning, raise a remediation issue; they are free, and can easily be closed again after additional input.

If it looks fine it probably is,
and even if it isn't then there's a rotating schedule of other engineers in the following weeks who can spot something you missed.

This process is designed to get warning of things that are coming when possible,
not to be a perfect predictor in all cases.

Resources

Resources used by the team to conduct work are described on the Development Resources Page.

History and trivia

- Name for Switchboard customer portal was suggested by @marin after he spent a day trying to figure out which mixing console (Also known as a switchboard/soundboard) to get for amateur music making. He didn't buy anything, but the suggestion was accepted.
- Name for Amp management cluster was suggested by @ccasella, as it is the instance that is "powering" supply of other instances.
- Dedicated Group - Year in Review 2023

SECTION: engineering/infrastructure/team/gitlab-dedicated/dedicated-migration-team.md

The Team

The Dedicate migration team is a team focussed on performing migrations into GitLab Dedicated. This is achieved using either Geo or Congregate.

Team members

Name	Role
---	-----
Muhammed Huseinba.i.	Senior Site Reliability Engineer
Dave Dittman	Site Reliability Engineer
Sophia Langer	Project Manager
Eddie Yoffe	Program Manager
Oliver Seldman	Program Manager
Eren Ekca	Senior Technical Architect
Saurav Issar	Senior Technical Architect
Petar Proki.	Senior Engineer

Jessykah Bird	Engineer
Jordan Ng	Engineer
Alex Rosencrans	Engineer
Scott Murray	Engineer
Douglas Alexandre	Staff Engineer
Sampath Ranasinghe	Senior Product Manager
Akanskha Joshi	Technical Program Manager

Mission

Successfully manage incoming migrations through FY26Q2 and FY26Q3 and establish a repeatable process for both Geo and Congregate migrations with a SSOT.

Resources

- GitLab group
- GitLab project
- Migrating Customers to Dedicated internal documentation

Process

SECTION: engineering/infrastructure/team/gitlab-dedicated/environment-automation.md

Summary

Environment Automation is a team within the Dedicated Group. Our mission is to develop and operate the automated plumbing of the GitLab Dedicated solution.

We follow the same processes as listed on the Dedicated Group, unless a difference exists which is explicitly noted on this page.

Team Members

{{< team-by-manager-slug "o-lluch" >}}

{{< team-by-manager-slug "denhams" >}}

Working with us

To engage with the Environment Automation team:

- Create an issue in the GitLab Dedicated team issue tracker
- Label the issue with:
 - workflow-infra::Triage
 - team::Environment Automation
- When creating an issue, it is not necessary to @mention anyone
- In case you want to get attention, use a specific team handle (Ex: @gitlab-dedicated/environment-automation) as defined in Dedicated group hierarchy

- Slack channels
 - For Environment Automation specific questions, you can find us in gdedicated-environment-automation-team
 - Our Slack group handle is @dedicated-envauto-team
 - The Dedicated Group as a whole leverages: gdedicated-team
 - Other teams in Dedicated group have their own work channels for team work discussions:
 - gdedicated-us-pubsec
 - gdedicated-switchboard-team

How We Work

Our preference is to work asynchronously, within our project issue tracker as described in the project management section.

The team also has a set of regular synchronous calls:

1. Environment Automation Team Sync (alternate weeks):
 1. EMEA/AMER: Tue 15:00 UTC (Good for EMEA and US East)
 1. PST/APAC: Wed 00:00 UTC (Good for APAC and US West)
1. Dedicated on GCP - Weekly Demo: Wed 07:30 UTC
1. Dedicated Group Demo

Reviewer roulette

Reviewer roulette is an internal tool for use on GitLab.com projects that randomly picks a maintainer + reviewer. Environment Automation uses it to spread the MR review workload. To do so:

1. Go to the reviewer roulette page.
1. Click on Spin the wheel.

See the full MR process.

Example responses

Here are some concrete examples of responses to capacity planning alerts.

- Removing a metric from capacity planning - Advanced search memory pressure does not follow long-term trends and was not a useful prediction.
It remains a metric that is alerted on if it exceeds practical limits.
- Remove saturation metric entirely - kubepoolcpu was incorrect in many cases, and difficult to get right.
It needed to be replaced with a different saturation metric (node-based CPU).
- Add Saturation metrics - Kubernetes PVCs were not being monitored at all, leading to near-miss incidents
- Fix the saturation metric - Advanced search disk was inaccurate and needed to be replaced with better promql expressions

SECTION: engineering/infrastructure/team/gitlab-dedicated/slas.md

GitLab Dedicated: Service Level Availability and Disaster Recovery Plan

Service Level Availability

Service Level Availability is the percentage of time that GitLab Dedicated ("Dedicated") is in an available state for use by a customer over a given calendar month. This document defines how Service Level Availability is calculated for Dedicated.

Scope

GitLab calculates Service Level Availability based on the available state of certain services and features provided with Dedicated. The services and features that are considered when calculating Service Level Availability include:

Service Features
----- -----
webservice GitLab Issues, Merge Requests, CI Job Logs Page; GitLab API; Git Push, Pull, Clone Operations over HTTPS
registry Container Registry HTTPS requests
gitlab-shell Git Push, Git Pull, Clone Operations over SSH

Service Level Availability Calculation

For each service and feature described above, GitLab measures two service level indicators ("SLIs"), as further described in <https://gitlab.com/gitlab-com/runbooks/-/tree/master/reference-architectures/get-hybrid-service-level-indicators>:

1. The Error SLI is an indication of requests that are successful, (i.e. not returning a 5xx error).
1. The Apdex SLI is an indicator of requests that complete with a satisfactory latency. Apdex is defined using the industry definition with two latency thresholds: satisfactory and tolerable. For Dedicated, satisfactory requests take less than 1s to complete, tolerable requests take less than 10s to complete.

Combining these two SLIs, GitLab scores each request as follows:

Server Error? Latency <= 1s Latency <= 10s Latency > 10s
--- --- ----- -----
No 1 0.5 0
Yes 0 0 0

Service Level Availability is then calculated using the following measurement:

For each calendar month, we calculate the sum of the combined SLI scores for all requests in that month, excluding any requests made during maintenance windows, and divide this by the total number of requests during that period (again, excluding maintenance windows).

Service Level Availability = the sum of qualifying SLIs for the entire month divided by the

total qualifying requests for the entire month. The Service Level Availability of GitLab Dedicated should meet or exceed the current Service Level Objective (defined below).

Current Service Level Objective

GitLab's current monthly service level objective for GitLab Dedicated is 99.5% (the "Service Level Objective").

Exclusions

Calculation of Service Level Availability does not include failures resulting from (1) misuses or misconfiguration of the applicable service or feature provided with Dedicated, (2) customer activity outside GitLab's terms of service, (3) components or services not defined in the Scope section (above), (4) factors outside of GitLab's reasonable control, such as force majeure events, or (5) Customer's or selected cloud hosting providers services, equipment or other technologies. Scheduled maintenance or urgent unscheduled system maintenance necessary to address critical issues (e.g. security vulnerabilities, data consistency issues, etc) are also not included in the calculation of Service Level Availability.

Disaster Recovery Plan

GitLab has developed a disaster recovery plan (the "Plan") to minimize the impact of a disaster or other emergency impacting a customer's access to and use of Dedicated

DR Scope

The Plan is scoped to disasters or other emergency events impacting the following:

1. Partial region outage (e.g. AZ failure)
2. Complete outage to primary region

The recovery time objective ("Recovery Time Objective" or "RTO") for the Plan is based on how long it takes to re-provision the required infrastructure and restore data from the most recently available backup.

The recovery point objective ("Recovery Point Objective" or "RPO") for the Plan is based on the frequency of snapshots across the data sources.

In order to receive RPO and RTO targets, customers must specify a primary and secondary region upon onboarding and these regions must be supported by GitLab Dedicated. The list of regions that are supported by GitLab Dedicated can be found [here](#).

If a customer uses the Bring your own domain feature, the customer must update its Domain Name System ("DNS") configuration to fully restore the Dedicated service ("DNS Update"). The time required to trigger and complete a DNS Update will not count towards the calculation of RTO under the Plan.

The RTO and RPO outlined in the Plan do not apply in the following circumstances:

1. If only a primary region was specified by a customer

2. If a secondary region is not supported by GitLab Dedicated
3. If the secondary region selected by a customer only provides for limited support

In all three circumstances above, GitLab will still make a good faith effort to recover pursuant to the Plan, but the stated RTO and RPO objectives will not apply.

GitLab regularly tests the Plan and will take all commercially reasonable efforts to ensure its success within the below RTO/RPO goals.

DR Exclusions

Events that have a more severe impact in a customer's access to and use of Dedicated, such as loss to both primary and secondary regions, global internet outages, or data corruption issues, are out of scope from the Plan. GitLab will still make a good faith effort to recover pursuant to the Plan, but the RTO and RPO goals of the Plan will not be considered.

Current RTO Target

8 hours

Current RPO Target

4 hours

SECTION: engineering/infrastructure/team/gitlab-dedicated/switchboard.md

Summary

Switchboard is a team within the Dedicated Group. Our mission is to empower external GitLab Dedicated customers to manage their tenant environments and reduce the operational overhead on the Environment Automation team so we can scale up the GitLab Dedicated offering. We follow the same processes as listed on the Dedicated Group, unless a difference exists which is explicitly noted on this page.

Resources

- Switchboard Direction Page
- Switchboard Demo Library

Team Members

{{< team-by-manager-slug "ashiel" >}}

Product Manager: Loryn Bortins
Technical Writer: Lysanne Pinto
Product Designer: Jesse Hoek

Working with us

To engage with the Switchboard team:

- Create an issue in the GitLab Dedicated team issue tracker
- Label the issue with:
 - component::Switchboard
 - workflow-infra::Triage
 - team::Switchboard
- When creating an issue, it is not necessary to @mention anyone
- In case you want to get attention please use the specific team handle, @gitlab-dedicated/switchboard, as defined in Dedicated group hierarchy
- As a cross-functional team Switchboard uses @gitlab-dedicated/switchboard/frontend-engineers and @gitlab-dedicated/switchboard/backend-engineers internally to seek input from engineers with specific expertise
- Switchboard issue board tracks all of the team's current work
- Slack channels
 - For Switchboard specific questions, you can find us in gdedicated-switchboard-team
 - Our Slack group handle is @dedicated-switchboard-team
 - Issues relevant to the wider Dedicated Group may be raised gdedicated-team
 - Other teams in Dedicated group have their own work channels for team work discussions:
 - gdedicated-environment-automation-team
 - gdedicated-us-pubsec

Requesting Access to the Switchboard application

- Create an Access Request specifying
 - the specific environment (Test / Beta / Production)
 - level of access required (Readonly, Support, Provisioner, Operator)
 - justification for the access
- Access & Provision Details for the application can be found in the Switchboard - GitLab Dedicated section of the Tech Stack

How we work

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our project issue tracker as described in the project management section.

The team does have a set of regular synchronous calls:

- Switchboard Sync - During this call, we are sharing important information for team-members day-to-day, as well as project items requiring a sync discussion
- 1-1s between the Individual Contributors and Engineering Managers

Impromptu Zoom meetings for discussing Switchboard work between individuals are created as needed.

It is expected that these meetings are private streamed, or recorded(1), and then uploaded to GitLab Unfiltered playlist.

The outcome of the call is shared in a persistent location (Slack is not persistent). This is

especially important as the team grows, because any decisions that are made in the early stage have will be questioned in the later stages when the team is larger.

1 Exceptions to the recording rule are: 1-1 calls, discussions around non-project work, and in cases where parties do not feel comfortable with recording or we cannot record due to the nature of content discussed. However, even with the exceptions, outcome of project related discussions need to be logged in a persistent location, such as the main issue tracker.

Tracking, Planning & Delivering Projects

Resources

- Switchboard team roadmap
- Switchboard team top-level epic
- Switchboard team issue board
- Switchboard technical writing board
- Dedicated Issue Tracker

Quarterly Planning

Quarterly planning is owned and driven by the Switchboard EM and PM.

1. EM or PM creates a quarterly planning issue using the Dedicated quarterly planning issue template.

- Set the issue title to Switchboard - FYXXQY planning where XX is the year and Y is the quarter

- Ideally planning should be kicked off at least a quarter in advance. The planning issue for the upcoming quarter should be created on or before the first day of the second month of the current quarter

2. EM & PM brainstorm asynchronously on the issue

3. During month 3 of the current quarter EM or PM

- Reach out to the team for input
- Ensure all dependencies are communicated with the relevant teams
- Prioritise the objectives
- Document the associated epics

4. EM creates a delivery epic for the quarter that will be used in Grand Reviews and updates the epic-summaries bot to reflect this change

Directly Responsible Individuals (DRIs)

Each and every project is owned and driven by a Directly Responsible Individual or DRI.

Though Switchboard is a cross-functional team the DRI is responsible for delivering the project output including UX design, product decisions, frontend and backend implementation along with any other deliverables, regardless of their expertise.

For example if the DRI is a backend engineer, while they may not directly implement every issue, they are still responsible for checking progress to ensure other team members have access to the information needed to fulfil the project.

The DRI will not directly implement every piece of the project but they are responsible for ensuring it is delivered. This involves collaborating with the EM to ensure issues are

prioritised within the engineering team, collaborating with the various functions within the team to ensure they have access to the information needed to fulfil the project (UX, frontend, backend, product etc), highlighting possible risks and ensuring alignment across the team.

We use issues to collaborate on ideas and solve problems. Everyone working on issues is responsible for keeping its description (SSoT) up to date with the latest decisions, and to ensure any follow-up resulting work is being tracked. The DRI is responsible for ensuring discussion threads reach a decision.

DRI's should also drive a pivot to synchronous discussion when a thread is not yielding a timely conclusion.

In the case where the UX design has been agreed upon prior to implementation, the DRI can accelerate the MR review process and reduce the workload on the PM and UX designer by ensuring the MR deliverable matches the design provided in Figma and approving accordingly.

The Epic Refinement process followed by the DRI at the beginning of the project is described below in the Epic Refinement section.

The DRI is also responsible for weekly status updates (further details) and ensuring the epic has a demo link attached (see Switchboard Demos).

The instructions for delivering the final status update and closing an epic can be found [here](#).

Epic Refinement

Switchboard team process to refine epics:

1. Identify a DRI for the epic
 - Team members can volunteer or the EM may ask team ask specific team members to act as DRI
1. Identify any missing requirements
 - All team members ask questions in the comments of this issue to drive out any edge cases
1. DRI labels the epic as ~"workflow-infra::Ready"
1. DRI ensures Epic Kick-Off is complete - see Epic Template
1. Assign Due Date & Start Date
 - DRI, EM & PM work together to assign due date based on team capacity, external deadlines and amount of work involved
1. DRI identifies at least one demo that will be delivered with the epic and adds a brief outline to the epic description (see Switchboard Demos).
1. For substantial code modifications or architectural additions, the DRI creates and shares draft technical documentation with team early, if possible, before development work starts.
 1. Draft documentation should contain a high level technical explanation of the new system and how it is meant to work
 1. Illustrative diagrams like sequence diagrams are encouraged
 1. Technical documentations can be added to the switchboard project where they currently live under ./docs directory
1. EM or DRI labels individual issues as ~"workflow-infra::Triage"
1. DRI enables issues to be worked on in parallel where possible so that multiple engineers can contribute to a single epic
1. If the epic involves both Frontend and Backend implementation the issues should be labeled accordingly
1. Team members pick up issues and start working on them
1. Team members use Progress Threads to track progress in individual issues
1. Team checks in on progress during Switchboard Sync

- Any epics with no issues to raise can be read-only
 - Priority given to discussions of epics based on soonest due date
1. If due date is not realistic team member comments on the issue as early as possible so that the team can work together to address this

Note 1, 2 & 4 can be carried out in parallel

Issue Refinement

Switchboard team process to refine issues:

1. When an issue is created and ready to be refined it is labeled ~"workflow-infra::Triage"
1. PM and EM ensure that the Open and ~"workflow-infra::Ready" columns are prioritised
1. Team members look at issues in the Open column of the issue board and ask questions on the issue to drive clarity
1. When there are no outstanding questions on the issue it can be labeled ~"workflow-infra::Ready" and it will automatically move into the Ready column
1. If the issue exposes text to users in any way the technical writing label should be added. For example if the issue changes UI text, shows an error message, adds a field etc
1. If the issue requires Frontend implementation the frontend label should be used
1. DRI enables issues to be worked on in parallel where possible so that multiple engineers can contribute to a single epic
1. The default is to keep both frontend and backend implementation for a single piece of functionality on the same issue so that discussions are centralised, implementation is carried out in parallel and frontend and backend engineers are in sync
1. If the implementation requires modification or creation of an API endpoint, a plan on endpoint, params structure and return data structure should be agreed upon between Frontend and Backend as early as possible to avoid re-work.
1. Frontend and backend implementation should be delivered in separate MRs
1. If the implementation cannot be done in parallel, or there is a likely to be a meaningful delay between backend and frontend implementations, or if the backend can deliver value independently the issue should be split and the relationship clearly identified by linking the issues
1. SRE dependencies should be called out as early as possible with requirements clearly documented in issues (example SRE dependency issue)

Issue & Epic tracking

1. Engineers use Progress Threads to share progress in an async fashion
1. At the beginning of the Switchboard Sync the team will check in on epics labeled ~"workflow-infra::In Progress" or ~"workflow-infra::Triage" to ensure due dates are appropriate and highlight any blockers
1. Epic DRIs update the status in the Epic Description every Wednesday in preparation for the Grand Review
1. Epic DRIs review the due date weekly. The epic status update should include the DRI's confidence level in the due date and any risks to delivery

Picking up work / What to work on next

1. ~"workflow-infra::Ready" column on the issue board

1. Pick an issue from the ~"workflow-infra::Ready" column on the issue board
 1. Assign the issue to yourself and set to ~"workflow-infra::In Progress"
 1. Update the issue description with an Implementation Plan where relevant
 1. Create a Progress Thread on the issue and update daily
1. Switchboard team top-level epic
 1. Look at the Switchboard top level epic and offer to work on issues with the nearest due date
 1. Use the Switchboard team roadmap for guidance
 1. Open column on the issue board
 1. Look at the issues at the top of the Open column
 1. For each issue ask questions and drive a discussion to identify any missing information
 1. Mark the issue as ~"workflow-infra::Ready" if it is ready to be worked on or @mention the EM if you are not confident to do so

Switchboard Demos

A demo enables a team member to share progress or the final output of an issue or epic. The focus is on sharing information, not creating an oscar winning documentary, so as a team we use the boring solution of either a screen recording, a recorded Zoom meeting or Loom. A link to the recording is added to the epic description.

The epic DRI is responsible for identifying at least one demo that will be delivered with the epic when the epic is being kicked off. For example When the functionality x is delivered (or issue y is Done) we will demo functionality x.

Team members are encouraged to time demos to be delivered shortly before the fortnightly Switchboard Team Syncs whenever possible so that any synchronous Q&A can happen during already reserved time.

Team members may choose to create additional demos to share progress or delivery milestones.

Issue Templates

Switchboard maintains the following issue templates:

Template	User	Use case
Further Details		
----- ----- -----		
----- -----		

switchboardbug.md	GitLab team members	Reporting a bug in the Switchboard application
Usually used by EA and Switchboard team members		
featureproposalswitchboard.md	GitLab team members	Proposing a new feature
Usually used by Switchboard team members. Could possibly be replaced by Epic template or updated to ensure more fields are optional		
switchboard-feedback.md	GitLab team members	This is used to make

suggestions and provide feedback to Switchboard. It feeds into the EA Requests epic which the EM & PM prioritise | This template is used regularly by EA team members to provide feedback

| switchboardtenantmodelschemaupdate.md | EA team members | Tenant Model Schema Updates

|

|

| switchboardtenantonboardingrequest.md | Onboarding DRI | Kicks off the Dedicated onboarding process

| Generally used by Dedicated PMs

|

| createonboardingtenantmodelrequest.md | Onboarding DRI | Used to track the creation of the OnboardingTenant in preparation for onboarding a new Dedicated customer

| Generally used by Dedicated PMs

|

| requestforswitchboardhelp.md | Support Engineers | Highlight an issue and request help from Switchboard team members

|

|

| switchboardteammemberonboarding.md | Switchboard EM | Onboard a new team member to the Switchboard team

|

|

| switchboardinternalissue.md | Switchboard team member | This template is used by DRIs to create issues in pre-existing well-defined epics

|

|

Merge Request Review Guidelines

We specifically adhere to the GitLab Code Review Guidelines and follow the Dedicated group principles when requesting merge request reviews.

Merge request review process

As the Switchboard team is currently small, we use an 'Approve and Merge' approach:

1. When you're ready to have your merge request reviewed, select one or more Switchboard reviewers.

- If you're not certain about who to choose, you can use the reviewer roulette to randomly select a reviewer.

- If the issue is labeled technical writing add the Switchboard technical writer as a reviewer

1. Reviewers will perform a review based on reviewing a merge request guidelines.

1. If satisfied, a reviewer will approve and merge unless other reviewers have questions or suggestions that are not addressed.

1. If the merge request contains the required approvals, the reviewer will trigger a pipeline and set auto-merge.

- If the reviewer does not have merge permission, they should seek out a maintainer for merging.

Additional UI Review Process

In addition to the above when a change is being proposed to the UI the following extra steps should be followed:

UI changes visible to internal GitLab users:

1. MR author cc's PM & UX Designer on the MR but they are not reviewers or blockers of the merge
1. If they have any suggestions they can be dealt with on the MR or on a later MR at the MR author's discretion
1. Eventually the PM & UX Designer will be reviewers on the internally visible UI updates but our process is not there yet nor is their capacity
1. If you require help or guidance with the UX or copy please ask before implementation begins on the issue

UI changes visible to external customers:

1. It is highly important to figure out any outstanding questions on the issue, including design changes, copy changes, and product requirements so that we can avoid ambiguity at the MR stage. This must translate to a concrete conclusion of design discussions in the issue before UI code changes are started.
1. In the event of a missing UX/product piece at the MR stage, the DRI and issue assignee will collaborate to make a decision about whether the missing piece can be resolved on the MR or if it needs to be moved back to the issue for resolution. This is in accordance to the DRI definition.
1. The PM and Designer will deal with these requests as a top priority.
1. Add the PM as reviewer on the MR to keep them updated. This review will not impede the MR progress, and the PM will handle it as a high priority. Ideally, product requirements should be finalized in the issue.
1. If there is a new copy change, add the Technical Writer as a reviewer to keep them informed and this will be non-blocking (copy should ideally be agreed on in the issue). Additionally add the Technical Writing tag to the MR.
1. Cc the UX Designer on the MR and when they are ready to be core reviewer this will be communicated to the team

Note: If significant discussion ends up being needed for a UI change (internal or customer facing) after the MR has been opened, that discussion should be moved back to the issue to resolve and the MR marked as blocked. These discussions will be high priority to resolve and the issue should be assigned to the PM and Designer until progress on the MR can resume.

Notes:

- It is our intention to move towards a typical 'reviewers and maintainers' approach which would require two reviews as soon as we have the team members to support this.
- Merge requests should be approved based on the approval guidelines.
- As per the GitLab Review Guidelines there are scenarios where it is appropriate for the author to merge the merge request: If there are no blocking comments, and the merge request has all the required approvals, the author or maintainer can merge.
- Switchboard project is configured to use pipelines for merged results which means that reviewers need to run a pipeline pre-merge to guarantee that updates are compatible with the

latest main branch.

- When reviewing merge requests, reviewers should use the Conventional Comment labels to convey your intent.

- For the avoidance of doubt Suggestion:, Issue: and Chore: comments are all blocking, unless decorated with a (non-blocking) statement.

- We label merge requests using the Specialization labels found in the GitLab documentation. MRs should be labeled ~"frontend", ~"backend" or ~"documentation"

Approval guidelines

If your merge request includes	It must be approved by a
-----	-----
~backend changes	Backend maintainer.
~frontend changes	Frontend maintainer.

Reviewer roulette

Reviewer roulette is an internal tool for use on GitLab.com projects that randomly picks a maintainer for each area of the codebase. To select a maintainer:

1. Go to the reviewer roulette page.

1. Select the Switchboard project.

1. Choose the desire role: ~maintainer::frontend, ~maintainer::backend, ~reviewer::backend, ~reviewer::frontend.

1. Click on Spin the wheel.

GitLab Code Review Guidelines

- Having your merge request reviewed
- Reviewing a merge request
- The Right Balance
- Quality
- Performance, reliability and availability
- Merge request performance guidelines

Reviewers and maintainers

There are two groups for Switchboard, Reviewers and Maintainers:

- All Switchboard team members are included in the Reviewer group.
- When a team member is fully onboarded and feel confident in their knowledge of the codebase they are invited to the Maintainer group.

Epic Template

<details><summary>Epic Template</summary>

markdown

DRI :levitate:

- TBC

Participants :bustinsilhouette:

- Frontend Engineer:
- Backend Engineer:
- SRE:
- UX Designer:
- PM:
- Technical Writer:
- EM:
-

Problem to solve :thinking:

Video Walkthrough from Product :videocamera:

Intended users :bustinsilhouette:

User	Visible	Description
Internal - Operator		
Internal - Support		
Internal - Provisioner		
Internal - Read Only		
External - Tenant Admin		
Internal - Read Only		

User experience goal :goal:

<!-- Overview of user experience goal -->

UX Design Spec :paintbrush:

- Figma Link:
- Dev mode:
- Any other details

Proposal :bulb:

Open Questions :question:

Question	Added by	Discussion thread	Resolved

Feature flags

<!-- This table should document any feature flags that were added or removed by the epic -->

Feature flag	Details

Further details :mag:

Dependencies :link:

Permissions and Security :link:

Documentation :book:

Publicly Accessible Documentation:

Epic Kick-Off :ballotboxwithcheck:

- ☐ Video walkthrough from Product outlining expectations
- ☐ DRI identified
- ☐ Roll out plan agreed
- ☐ External customer communication plan defined
- ☐ Copy Requirements are highlighted to the Technical Writer
- ☐ UX Requirements are highlighted UX Designer
- ☐ Issue created to track Documentation requirements
- ☐ Outstanding Questions captured in threads for resolution
- ☐ Frontend, Backend and SRE dependencies highlighted and / or unknowns highlighted

Roll out Plan :speaker:

<!--

If visible to external customers please provide the following information:

- What communication is required ahead of release?
 - ☐ Internal communication to account teams
 - ☐ Customer communication in release post
 - ☐ Sign off from account teams before release - this should be reserved for features

with the potential to be disruptive to users

- Will this be rolled out to customers in pieces as implemented or available internally first?

-->

Links / references :books:

Demo Description :moviecamera:

Demo Link - see <https://handbook.gitlab.com/handbook/engineering/infrastructure/team/gitlab-dedicated/switchboard/switchboard-demos>

<!-- STATUS NOTE START -->

<!-- STATUS NOTE END -->

/label ~"team::Switchboard" ~"workflow-infra::Triage"
/confidential

</details>

SECTION: engineering/infrastructure/team/gitlab-dedicated/us-public-sector-services.md

Mission

The GitLab US Public Sector Services team mission is to create a fully managed, single-tenant GitLab environment, served through a GitLab Dedicated platform that is purpose-built to help address specific regulatory and compliance requirements of US government agencies at the federal, state, and local level, as well as contractors, educational institutions, and other U.S. customers that run sensitive workloads. It is developed to remove any manual interactions with customer tenant installations, and to ensure that the customer tenants are fully focused on unlocking the power of The One DevOps Platform.

Vision

The US Public Sector Services group is a customer facing team, with team members focused on a high level of infrastructure automation, and enabling customer interactions with the GitLab Dedicated for US Government platform.

Team mission is to:

- Build cloud infrastructure that meets or exceeds the requirements of the Federal Risk and Authorization Management Program (FedRAMP)
- Develop a 100% automated system for provisioning a large number of single tenant GitLab sites
- Automate maintenance tasks without human interaction
- Create and manage central observability stack, as well as observability stack per customer tenant
- Create customer portal (Switchboard), exposing administrative operations to customer tenants

Performance Indicators

Team performance indicators are not fully defined. We are going to consider a Provisioning SLO to start with, possibly followed by DORA 4 metrics.

Team Members

{{< team-by-manager-slug "mckgl" >}}

Working with us

To engage with the GitLab US Public Sector Services team:

- Create an issue in the GitLab Dedicated team issue tracker (Note The issue tracker will be made public after work in epic 33 "Considerations for making Dedicated projects public" is completed). Label the issue with team::US PubSec label
- When creating an issue, it is not necessary to @mention anyone
- In case you want to get attention, use a specific team handle as defined in group hierarchy below
- Slack channels
 - For GitLab US Public Sector specific questions, you can find us in gdedicated-us-pubsec
 - The @dedicated-uspubsec-team Slack group can be used in any internal channel to tag the team.
 - Issues relevant to the wider Dedicated Group may be raised in gdedicated-team
 - Other teams in Dedicated group have their own work channels for team work discussions:
 - gdedicated-environment-automation-team
 - gdedicated-switchboard-team

How we work

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our project issue tracker as described in the project management section.

The team does have a set of regular synchronous calls:

- Team call - During this call, we are sharing important information for team-members day-to-day, as well as project items requiring a sync discussion
- 1-1s between the Individual Contributors and Engineering Managers

Impromptu Zoom meetings for discussing GitLab Dedicated work between individuals are created as needed.

It is expected that these meetings are private streamed, or recorded⁽¹⁾, and then uploaded to GitLab Unfiltered playlist.

The outcome of the call is shared in a persistent location (Slack is not persistent). This is especially important as the team grows, because any decisions that are made in the early stage have will be questioned in the later stages when the team is larger.

1 Exceptions to the recording rule are: 1-1 calls, discussions around non-project work, cases where parties do not feel comfortable with recording, or we cannot record due to the nature of content discussed. However, even with the exceptions, outcome of project related discussions need to be logged in a persistent location, such as the main issue tracker.

GitLab Group Hierarchy

We use GitLab Groups to logically organize team-members working on GitLab Dedicated projects. The groups cover the following use-cases:

1. GitLab US Public Sector Services group membership: @gitlab-dedicated/uspubsec
 - All permanent team-members in any of the GitLab Dedicated teams should gain access to this GitLab group as part of onboarding
 - Group mention should only be used in circumstances where the information shared is

pertinent for all team members of GitLab Dedicated group

2. Individual team GitLab groups have two additional subgroups maintainers and reviewers, for example: @gitlab-dedicated/uspubsec/maintainers

- reviewers GitLab group access is granted to permanent team-members, external contractors, team-members on borrows and similar. This GitLab group type is used to distinguish users without merge rights. Initial reviews should be requested from this group, using the quick action, for example. /assignreviewer @gitlab-dedicated/uspubsec/reviewers

- maintainers GitLab group is granted to permanent team-members only. This group has merge rights, and the group is granted access through CODEOWNERS approval rules

Do note that the transition between reviewers and maintainers groups is still not fully defined, and this will be defined as the team continues to grow. Currently, the founding team-members with a significant early contribution to the specific codebase and specific topic knowledge have been granted maintainers rights.

Project Management

We use epics, issues, and issue boards to organize our work, as they complement each other.

The single source of truth for all GitLab US Public Sector Services work across different functions is the top-level GitLab US Public Sector Services epic. Please view that epic for more details on active and upcoming work. To view the specific issues the team is working on, see the overall issue board for the team's work.

Epic Hierarchy

Sub-epics are created under the top-level epic to logically segment work into an organized list of issues that are targeted towards a specific initiative or project milestone.

When applicable, additional sub-epics may be created within the existing epic hierarchy to further segment issues for project tracking purposes.

1. Sub-epics group tasks required to deliver an item mentioned
2. Sub-epics represent an item from the roadmap and are delivered in a specific phase
3. Sub-epics can span multiple months, but their end date should match the 'anticipated completion date' of the roadmap phase they are added to.

In addition to epics, milestones can be used as a project tracking tool that allows issues across multiple epics to be associated with a target milestone date. Milestones can be filtered on an issue board for a single-pane view of current priority issues to be actioned by the team.

Epic Owners

Each epic has a single DRI who is responsible for delivering the project. DRIs for each epic are listed at the top of the description of each epic per Epic Structure.

Epic Owner Responsibilities

The DRI needs to:

1. Work with others to move issues through the boards
2. Ensure epic meets criteria outlined in Epic Structure
3. Provide updates on DRI's epic in epic description according to process outlined in Status Update Process below.

Epic structure

Each epic and child sub-epics must include the following:

Description

1. DRI who is responsible for this epic.
2. Background, including a problem statement, to provide context for people looking to understand the epic.
3. Exit criteria for the specific goals of the epic.
4. Status yyyy-mm-dd should be the final heading in the description.
 1. This enables others who are interested in the epic to see the latest status without having to read through all comments or issues attached to the epic.
 2. This heading is used to auto-generate the status information on the top-level epic.

Epic meta data

1. Start date is set to the expected start date, and updated to be the actual start date when the project begins.
2. Due date is set to be the expected end date.

Labels are described in the epic label section.

Issue boards

Issue boards are used to track the overall status of epics and/or milestones.

Accessing US Public Sector Services Issue Boards

1. Navigate to the Dedicated Issue Tracker project.
2. Using the navigation menu on the left, hover over the Issues section and select Boards
3. Using the drop-down menu located in the upper-left (next to the Search Filter) select the specific board associated with the epic or milestone that you want to view. If you know the name of the board you want to view, you can type it in the drop-down search box.
4. Once selected, the issue board will contain a kanban layout that contains a list of issues based on the filter provided (ex. milestone, epic, label, etc.), as well as various columns (also referred to as Lists) to reflect the desired organization of the issues. The US Public Sector Services team uses labels as the primary List filter. See the section below for how these issue boards are created and filtered.

> Note: Issue boards used for time-blocking project work should be deprecated once complete to maintain simplicity.

Execution

The team operates in a kanban fashion. Issues are prioritized in the kanban board and self-assigned. We leverage scoped workflow labels to track different stage of work.

Status Updates

The status for all work for the team is maintained in the description of the top-level GitLab US Public Sector Services epic so that it is visible at a glance.

Status Update Process

Both Engineering Cross-Functional DRIs should provide weekly updates for the DRI's epics according to following process:

1. By 17:00 PST on Thursdays DRIs of active epics (or the person covering if DRI OOO) provide an update in the status section of the description of the epic regarding status of the epic including any relevant details of active sub-epics.

- If the DRI for a sub-epic is different than the epic DRI, the epic DRI is responsible for getting updates from the sub-epic DRI.

- Format for weekly update:

- Date of Update (YYYY-MM-DD)

- Brief update (~sentence or couple bullets) for each of these three bullets:

- Progress since last update - Changes deployed to production, unblocked blockers, any other progress achieved.

- Risk and Confidence - Any new blockers identified or existing blockers that persist? Any other challenges now or in the near future? How do these blockers and/or challenges affect our confidence of completing by scheduled due date from Phase timeline?

- Mitigations - What is required to overcome challenges or blockers identified?

Should this be escalated to other team members, teams, executives, or domain experts?

- Update Workflow and Health label - After each status update, the Workflow label and Health label should be updated. See Epic labels criteria

2. Top-Level Epic Status Update automation synthesizes updates from status section from description of active epics to provide initiative status in the status section in the description of the top-level initiative Epic.

3. Weekly engineering/product sync at 16:30 UTC / 11:30 AM ET on Mondays Dedicated engineering/product meeting is used to discuss status updates and potential mitigations as necessary.

4. Status updates will be incorporated into initiative status updates and any initiative reporting in the following week.

5. By 23:00 UTC / 5:00 PM ET on Wednesdays Initiative DRI will share a summary of major items from the weekly initiative update in ceo Slack Channel and link to the full update in the Initiative epic.

Status Update Automation

Status updates are auto-generated and added to description of GitLab Dedicated top-level epic using a bot running the epic issues summary project.

If no update has been provided in an epic or issue for over a week, the issue will automatically receive workflow-infra::stalled label. Engineering managers are responsible for reviewing the status of the issue and helping it move along.

Reporting

We provide reports on status of GitLab Dedicated to meet Top Cross-Functional Initiative requirements.

SECTION: engineering/infrastructure/team/gitlab-dedicated/architecture/_index.md

Architecture

The architecture of GitLab Dedicated is maintained in GitLab Dedicated Architecture

Documents

- From Dedicated to Cells: a Technical Analysis

SECTION: engineering/infrastructure/team/gitlab-dedicated/architecture/from-dedicated-to-cells-technical-analysis.md

Introduction

This document is intended to be a high-level summary describing how the existing GitLab Dedicated architecture could be used as a foundation for deployment and operating large numbers of GitLab Cell instances.

Additionally, it explains certain decisions in the Dedicated architecture, and why these are beneficial to both GitLab Dedicated and GitLab Cells, and how they differ from other approaches, such as GitLab.com's IaC, or GET.

Note to readers: Please keep in mind that this is not a critique of GitLab.com's approach, nor GET's.

It is an explanation of how and why Dedicated differs from these other approaches, and offers some insight into how different requirements led to different design choices.

GitLab Dedicated is an Environment Automation Technology

In the context of this document, GitLab Dedicated can primarily be thought of as a technology to automate the configuration, deployment, upgrading, monitoring, and maintenance of a large number of mostly homogeneous GitLab application environments.

How does Dedicated differ from GitLab.com?

GitLab.com is the largest single GitLab deployment, and in all likelihood, the most complex. GitLab.com is provisioned and configured across multiple projects, including chef-repo for Chef Cookbooks for VM configuration, config-mgmt for Terraform configuration and the k8s-workloads group for Kubernetes configuration including Helm, Kubernetes manifests and Tanka.

Heterogeneous vs Homogeneous Environments

Let's discuss the distinction between homogeneous and heterogeneous GitLab environments.

GitLab.com: Many Heterogeneous Environments

At present, there are 64 environments managed in the config-mgmt project. In chef-repo, there are 22 Chef environments and 569 roles (some of these roles are likely unused). The GitLab.com helm charts are homogeneous, using the same Helm templates. For Terraform and Chef, each of these environments is unique, although they share many submodules and some effort is made to ensure consistency. However, each environment can differ from the others. There is no way to ensure uniformity, and over time these environments have, for a variety of reasons, drifted away from uniformity.

Similar but different: environments in GitLab.com's config-mgmt project

Since GitLab.com was originally built for a small number of environments (production and staging), it was not designed to be scaled up to a large number of homogeneous environments.

Another way of thinking about this is that the GitLab.com Staging and Production Terraform environments, gstg and grpd, are not replicas of one another, or the same configuration with different input values (for example to allow Staging to a smaller/cheaper copy of Production). They are two distinctive environments, built in parallel to one another.

```
<p float="middle">
  
  
</p>
```

Same Same, but Different: the GKE Zonal Configuration for Staging and Production. Similar, but

not the same.

GitLab.com evolved in this way for many reasons, and its approach allows for maximum flexibility. However, creating new environments takes weeks or months of operator time. Dedicated takes minutes of operator time and hours of wall time to create new environments. Additionally, from an operational and day-two point of view, the heterogeneous nature of these environments makes applying batch operations across many environments difficult, resulting in manual processes.

The non-heterogeneous nature of GitLab.com environments is further complicated by the multiple projects in which changes need to be coordinated, but more on this later.

Dedicated: Homogenous, repeatable, functional environments

Dedicated enforces homogeneity between environments by using a single IaC configuration (be it Terraform, Ansible, Kubernetes Manifests, etc) for all environments. The only differences that can exist between environments is controlled by the input variables to the environment. We call these input variables the tenant model. Since all environments use the same IaC, it is not possible to add or remove cloud resources, ansible configuration, terraform modules etc without those differences being expressed in the tenant model.

Having homogeneous environments provides greater consistency. This is especially important when the need to migrate many environments en-masse needs to take place, as there are far fewer possible combinations of configuration, allowing automation of changes.

IaC Configuration Tooling

GitLab.com: Many Different Configuration Management Tools

For GitLab.com, environment changes need to be coordinated and applied across a variety of different projects. Usually, manual coordination needs to be carried out to ensure that change is done in the correct order and sequencing. For example, Terraform changes may need to be applied, followed by chef-repo changes. If differences happen to occur in the manual coordination of these changes, it may lead to unexpected outcomes, ultimately leading to slower delivery, outages or even data-loss.

GitLab.com: Many Different Configuration Management Tools

Dedicated: Single Entrypoint for All Environment Changes

Dedicated seeks to simplify this model by providing a single project, Instrumentor, which

encapsulates all atomic changes to the tenant or cell environment. The problem of coordinating changes across multiple projects is solved by integrating all changes into a single project.

Dedicated: Single Entrypoint for All Environment Changes

Dedicated: an Atomic Provisioning Process Improves Determinism, Testability, and Repeatability

Freed from having to coordinate roll-outs across combinations of environments across different configuration projects, Dedicated allows environments to be understood in a far more deterministic manner. This in turn leads to greatly improved testability, repeatability and, when running many dozens or hundreds of environments, ultimately, better availability and resilience.

Compare a complicated database change for GitLab.com to a similar change in Dedicated. To make a change to a GitLab.com environment, one needs to:

1. Make Terraform module change for Staging
2. Make chef-repo change for Staging
3. Merge and apply Terraform module change for Staging
4. Merge and apply chef-repo change for Staging
5. Test change to ensure it does what it's expected to do
6. Create a similar, but not identical Terraform module change for Production
7. Merge and apply Terraform module change for Production
8. Merge and apply Chef-Repo change for Production

This example only describes two environments, but the more environments (eg, Pre-Prod) the more work to roll-out the change.

What's more: there's no guarantee that the environments are identical, or that some feature of one environment may change the behaviour in that environment. The environments are not deterministic and this leads to additional risk during change rollouts. This additional risk is then gated with a Change Management process in the form of CR's for more complicated changes. As is usual with manual steps, this process aims to reduce a chance of configuration drift but it also slows down the roll-out.

For Dedicated, the workflow is as follows:

1. Make a change in Instrumentor, testing the change in your sandbox environment using a review app on your merge-request branch.
2. Have change approved and merged to Instrumentor main branch.
3. Instrumentor will automatically release a new version of the Instrumentor package.
4. Starting with staging, progressively update all tenant models to the new version, and roll out across the fleet until all tenants/cells are running the latest version. Halt the rollout if problems are detected.

What's important is that for Dedicated, every change is encapsulated in a single lineage of packaged releases on a single artefact - Instrumentor.

Summary

	GitLab.com	GitLab Dedicated	
	-----	-----	-----

	IaC Configurations Does each environment share a single configuration or have its own unique TF module?		
		Many	One
	Provisioning/Configuration Repositories To make a wide-ranging Infrastructure change, how many projects does the change need to be spread across?		
		Many	One
	Flexibility in Configuration How flexible is the configuration, for example, for single environment deltas?		
(Tenant Model only)		Extremely High	Limited
	Day Two Operation Automation The likelihood of day-two operations being automated.		
	Sometimes Automated	Only Automated	

How does Dedicated differ from GET?

GET is a collection of tools to enable the deployment and operation of a base production GitLab environment at scale based on our Reference Architectures that can be built upon accordingly.

Dedicated uses GET under the hood but GET is primarily intended to be used for single instances of GitLab.

Additionally, GET is intended to be a general purpose GitLab environment suitable for general-purpose use cases. Importantly, GET does not (nor, should not) handle every possible use-case, such as many of the compliance, security, regulatory and advanced networking features provided by Dedicated.

GET rightly focuses on customers looking to maintain a multi-host and/or Kubernetes based GitLab install, running within a cloud environment, and adhering to a GitLab Reference Architecture. It does not focus on scaling this environment up to dozens or hundreds of instances, and adding the features to support this would add a great deal of complexity that would be a barrier for adoption for all GET users besides Dedicated.

The Components of GitLab Dedicated

This section describes the components of GitLab Dedicated, focusing on why these components were included in the design.

GitLab Dedicated Components

Switchboard

Switchboard is the user-interface for GitLab Dedicated customers, allowing customers to provision and configure their GitLab instances through a web user interface. This is analogous to Google Console for GCP access, Elastic Cloud for Elastic provisioning, etc.

Switchboard has always been focused on being the user-interface for GitLab Dedicated tenant operators; be that the SRE supporting the customer instance, or the customer operator. It is a product feature that simplifies customer interaction with GitLab Dedicated.

The design of Switchboard has only ever focused on the case of Dedicated, and in the opinion of the Dedicated team, allowing for customer interaction with their Dedicated instance. Cells was not in scope of GitLab Dedicated when Switchboard was designed, and Switchboard is only designed to make onboarding, interacting and information gathering simpler for the GitLab Dedicated customer.

Using S